

Actividad Practica Sistema de Gestion de Blog con API REST y Relaciones JPA

Objetivos de la Actividad

Al finalizar esta actividad, los estudiantes deberan ser capaces de:

1. Diseñar e implementar un modelo de datos relacional usando JPA con anotaciones como Entity, OneToMany y ManyToOne.
 2. Construir una API REST funcional con endpoints CRUD completos usando Spring Boot.
 3. Aplicar el patron DTO para controlar la informacion que se expone en la API.
 4. Configurar correctamente la capa de persistencia con JpaRepository y el archivo application.properties.
 5. Demostrar el funcionamiento de la API mediante pruebas en Postman.
 6. Resolver problemas comunes como la serializacion de relaciones JPA, evitando bucles infinitos.
-

Descripcion del Proyecto

Los estudiantes deben construir un sistema de gestion de un Blog que permita administrar Autores, Posts y Comentarios.

Modelo de Datos Requerido

El sistema debe manejar las siguientes entidades y sus relaciones:

Entidad Author:

- Atributos: identificador numerico autoincrementable, nombre (texto obligatorio), correo electronico (texto obligatorio y unico).
- Relacion: Un autor puede tener muchos posts. Esto es una relacion de uno a muchos.

Entidad Post:

- Atributos: identificador numerico autoincrementable, titulo (texto obligatorio), contenido (texto largo obligatorio), fecha de creacion (fecha y hora obligatoria).
- Relaciones: Un post pertenece a un autor. Esto es una relacion de muchos a uno. Un post puede tener muchos comentarios. Esto es una relacion de uno a muchos.

Entidad Comment:

- Atributos: identificador numerico autoincrementable, texto (texto largo obligatorio), fecha de creacion (fecha y hora obligatoria).
 - Relacion: Un comentario pertenece a un post. Esto es una relacion de muchos a uno.
-

Requerimientos Tecnicos

Capas a Implementar

Modelo: Crear las 3 clases de entidad (Author, Post, Comment) con las anotaciones JPA correctas. Para las relaciones, usar FetchType.LAZY para optimizar el rendimiento.

DTOs: Crear DTOs para cada entidad (AuthorDTO, PostDTO, CommentDTO) con metodos fromEntity() y toEntity(). Los DTOs deben incluir solo los atributos necesarios para la transferencia de datos, evitando exponer las relaciones completas que podrian causar bucles infinitos.

Repositorios: Crear interfaces para cada entidad que extiendan JpaRepository. Agregar metodos personalizados como buscar autor por correo electronico, buscar posts por identificador de autor, y buscar comentarios por identificador de post.

Servicios: Implementar la logica de negocio para cada entidad con metodos CRUD (findAll, findById, save, update, deleteById). Usar la anotacion Transactional en los metodos que modifican la base de datos. Verificar la existencia de las entidades antes de actualizar o eliminar, lanzando excepciones con mensajes claros cuando no se encuentren.

Controladores: Exponer endpoints REST para cada entidad con los metodos HTTP correspondientes. Usar RestController y RequestMapping con rutas claras y semanticas. Inyectar los Services mediante el constructor. Usar DTOs para las peticiones y respuestas, nunca devolver las entidades directamente.

Endpoints que Debe Exponer la API

Para Autor:

- GET /api/authors: Listar todos los autores.
- GET /api/authors/{id}: Obtener un autor por su identificador, incluyendo sus posts.
- POST /api/authors: Crear un nuevo autor.
- PUT /api/authors/{id}: Actualizar un autor existente.
- DELETE /api/authors/{id}: Eliminar un autor y sus posts en cascada.

Para Post:

- GET /api/posts: Listar todos los posts.
- GET /api/posts/{id}: Obtener un post por su identificador, incluyendo autor y comentarios.
- GET /api/posts/author/{authorId}: Listar todos los posts de un autor especifico.
- POST /api/posts: Crear un nuevo post, asociado a un autor existente mediante su identificador.
- PUT /api/posts/{id}: Actualizar un post existente.
- DELETE /api/posts/{id}: Eliminar un post y sus comentarios en cascada.

Para Comentario:

- GET /api/comments: Listar todos los comentarios.
- GET /api/comments/{id}: Obtener un comentario por su identificador.
- GET /api/comments/post/{postId}: Listar todos los comentarios de un post especifico.
- POST /api/comments: Crear un nuevo comentario, asociado a un post existente mediante su identificador.
- PUT /api/comments/{id}: Actualizar un comentario existente.
- DELETE /api/comments/{id}: Eliminar un comentario.

Guia de Implementacion para los Estudiantes

Configuracion Inicial

1. Crear el proyecto en Spring Initializr con las dependencias: Spring Web, Spring Data JPA, MySQL Driver, Spring Boot DevTools.
2. Configurar la base de datos en el archivo application.properties, incluyendo la URL, usuario, contraseña, dialecto de Hibernate y la propiedad ddl-auto con el valor update.
3. Crear la base de datos en MySQL ejecutando el comando CREATE DATABASE blog_api.

Creacion de Entidades

1. Crear la entidad Author con todos los atributos, la relacion OneToMany con Post y un metodo auxiliar para agregar posts.
2. Crear la entidad Post con todos los atributos, las relaciones ManyToOne con Author y OneToMany con Comment, y un metodo auxiliar para agregar comentarios.
3. Crear la entidad Comment con todos los atributos y la relacion ManyToOne con Post.
4. En todas las relaciones, usar FetchType.LAZY para evitar la carga innecesaria de datos.
5. Usar CascadeType.ALL en las relaciones OneToMany para que las operaciones en cascada funcionen correctamente.

Creacion de DTOs

1. Crear DTOs para cada entidad: AuthorDTO, PostDTO, CommentDTO.
2. Cada DTO debe tener constructores, getters y setters.
3. Cada DTO debe tener metodos estaticos fromEntity() para convertir de entidad a DTO y toEntity() para convertir de DTO a entidad.
4. En los DTOs, evitar incluir referencias circulares. Por ejemplo, un PostDTO puede incluir el autor mediante un AuthorDTO simplificado (sin lista de posts) o solo el identificador del autor.

5. Para el caso de PostDTO, incluir la lista de CommentDTO sin que estos comentarios tengan referencia al post.

Creacion de Repositories

1. Crear interfaces que extiendan JpaRepository para cada entidad, especificando el tipo de entidad y el tipo de su identificador (Long).
2. Agregar metodos personalizados utilizando la nomenclatura de Spring Data JPA, por ejemplo:
 - findByEmail(String email) para buscar autor por correo.
 - findByAuthorId(Long authorId) para buscar posts por identificador de autor.
 - findByPostId(Long postId) para buscar comentarios por identificador de post.

Creacion de Services

1. Cada Service debe manejar la logica de negocio de su entidad correspondiente.
2. Usar la anotacion Transactional en los metodos que modifican la base de datos (save, update, delete).
3. En los metodos findById, update y delete, verificar que la entidad exista antes de continuar. Si no existe, lanzar una excepcion con un mensaje claro.
4. En el metodo save de Post, verificar que el autor exista antes de crear el post. Si no existe, lanzar una excepcion.
5. En el metodo save de Comment, verificar que el post exista antes de crear el comentario. Si no existe, lanzar una excepcion.
6. Inyectar los Repositories mediante el constructor.

Creacion de Controllers

1. Usar RestController y RequestMapping con rutas claras y semanticas (/api/authors, /api/posts, /api/comments).
2. Inyectar los Services mediante el constructor.
3. Crear endpoints para cada operacion CRUD, usando DTOs para las peticiones y respuestas.
4. En los metodos GET, devolver DTOs convertidos desde las entidades.
5. En los metodos POST y PUT, recibir DTOs, convertirlos a entidades, procesarlos y devolver el DTO resultante.

6. En los metodos DELETE, no devolver contenido (void).

Configuracion Adicional

Configurar CORS con una clase CorsConfig que implemente WebMvcConfigurer, permitiendo todos los origenes para desarrollo. Esto es opcional pero recomendable para futuras conexiones con un frontend.

Pruebas en Postman

1. Probar cada endpoint, incluyendo los flujos completos: crear autor, crear posts para ese autor, crear comentarios para esos posts, modificar y eliminar cada elemento.
 2. Verificar que los DTOs no incluyan relaciones recursivas que causen bucles infinitos al serializar a JSON.
 3. Probar casos de error, como intentar crear un post con un autor inexistente o eliminar un autor que tiene posts sin cascada.
-

Entregables

Los estudiantes deben entregar un repositorio de GitHub que contenga:

1. El codigo fuente completo del proyecto Spring Boot en su estado final.
 2. Capturas de Postman que muestren cada endpoint probado (GET, POST, PUT, DELETE) con sus respectivas respuestas.
 3. Captura de MySQL que verifique que los datos se guardaron correctamente en la base de datos.
-

Consejos

Empieza por el modelo: Crea las entidades y sus relaciones antes de escribir cualquier otra cosa. Asegurate de que las relaciones bidireccionales tengan mappedBy en el lado adecuado.

Prueba cada endpoint: Con Postman, verifica que la API funcione antes de pasar al siguiente endpoint. No dejes las pruebas para el final.

Evita bucles infinitos: En los DTOs, no incluyas referencias circulares. Un PostDTO no debe tener un AuthorDTO que a su vez tenga una lista de PostDTO. Usa DTOs simplificados o solo identificadores para las relaciones.

Usa ddl-auto=update: Durante el desarrollo, usa esta propiedad para que JPA cree y actualice las tablas automaticamente. En produccion, usa none o validate.

Revisa los logs: Si algo falla, mira la consola de IntelliJ para ver las consultas SQL que esta ejecutando JPA y los errores que se estan produciendo.

No te saltes pasos: La actividad esta diseñada para que los estudiantes construyan el sistema paso a paso. Si te saltas un paso, es probable que los siguientes no funcionen.

Pregunta si tienes dudas: Si algo no funciona o no lo entiendes, pregunta. Es mejor preguntar que quedarse con dudas.
